A Dual-Memory Policy for Long-Horizon Counting Manipulation

Hefeifei Jiang
EE265B Final Project
University of California, Riverside
hjian105@ucr.edu

Abstract: Long-horizon manipulation tasks are often non-Markovian: the current observation alone cannot tell a robot how far it has progressed. Counting tasks expose this dependence in its purest form: a policy must repeat a sub-task exactly N times and then stop, while every repetition leaves the scene looking nearly identical. The recent RoboMME benchmark shows that no single memory representation solves such tasks: symbolic subgoals carry the count explicitly but rely on an error-prone vision–language model (VLM), while perceptual memory is robust but never represents the count directly. We propose a dual-memory vision–language–action (VLA) policy that combines both pathways and uses perceptual memory to correct the symbolic pathway: a learned token-wise gate suppresses subgoal information when it conflicts with visual evidence, and a stale-subgoal augmentation teaches the policy to recognize outdated subgoals during training. On the PickXTimes counting task, our experiments surface three findings that we argue matter more than the raw success rates: (i) VLM-generated subgoals fail in a structured way, drifting and repeating stale stages; (ii) perceptual memory is most valuable as a verification signal layered on top of symbolic memory rather than as a replacement for it; and (iii) VLM-based verification is effective but computationally expensive, motivating lightweight learned verifiers as a path forward.

Keywords: Memory, Vision-Language-Action Models, Long-Horizon Manipulation

1 Introduction

Most robot manipulation research targets short-horizon skills such as grasping a cup or inserting a plug, where a single observation contains everything the policy needs. Long-horizon tasks break this assumption: when a robot must repeat a pick-and-place motion N times, progress stretches across hundreds of control steps and the task becomes non-Markovian: the current camera frame no longer reveals where in the task the robot is, so the policy has to remember its own progress.

Counting tasks isolate this difficulty in its cleanest form. In the PickXTimes task of the RoboMME benchmark [1], a robot must pick and place a cube a specified number of times and then press a button to stop. After every repetition the scene returns to a visually near-identical configuration; the only quantity separating "done twice" from "done three times" is an internal count k that exists nowhere in the pixels. Success is binary and interpretable, which makes counting an ideal probe for studying memory in isolation.

RoboMME equips a shared $\pi_{0.5}$ backbone [2] with three families of memory representation and evaluates each in isolation: symbolic memory stores history as language subgoals written by an auxiliary VLM, perceptual memory stores compressed visual tokens from past frames,

and recurrent memory compresses history into a fixed-size latent. Its central finding is that no single representation dominates; each carries the count k in a differently flawed way. Symbolic memory can state the count explicitly ("placed 2 of 3") but inherits the instability of the VLM that writes it. Perceptual memory is end-to-end and needs no external model, but the count is never written anywhere and must instead emerge from a stack of near-identical frames.

These failure modes are complementary, which suggests a question RoboMME leaves open: what happens when the two representations are combined, and can one repair the other? This report answers it on PickXTimes with a dual-memory VLA policy: symbolic grounded subgoals enter through the language prompt; perceptual frame-sampling memory modulates the action expert; and a learned token-wise correction gate lets visual evidence suppress the symbolic pathway when the subgoal is wrong. Because the dominant VLM failure we observed is staleness, that is, re-emitting a subgoal from several steps ago, we additionally inject outdated subgoals during training, teaching the policy what an untrustworthy subgoal looks like.

We make three contributions: (1) a dual-memory architecture on RoboMME: to our knowledge the first to combine symbolic and perceptual representations in this framework, where all fourteen released variants are single-representation; each pathway keeps its strongest integration mechanism, two independently trained checkpoints are merged, and only lightweight adapters are fine-tuned; (2) a correction mechanism for unreliable subgoals: a token-wise sigmoid gate plus a stale-subgoal augmentation that simulates the VLM's real failure distribution; and (3) an empirical analysis of why the symbolic pathway fails: characterizing subgoal drift and stale repetition, showing perceptual information works best as a verification layer, and quantifying the cost of VLM-based verification. We consider the third contribution the most transferable.

2 Related Work

Vision–language–action models. Large VLA policies map observations and language instructions directly to actions: OpenVLA [3] and the $\pi_0/\pi_{0.5}$ family [4, 2] couple a pretrained vision–language backbone with a flow-matching action expert, building on action-chunking imitation learners such as ACT [5] and Diffusion Policy [6]. These models are predominantly Markovian and struggle on history-dependent tasks, which is exactly the setting studied here.

Memory for manipulation policies. MemoryVLA [7] maintains a perceptual–cognitive memory bank; MemER [8] retrieves keyframes from episodic experience; RMBench [9] classifies tasks by how many past observations must be retained; recurrent mechanisms (RMT [10], test-time training [11]) compress history into fixed-size states but proved unstable when grafted onto a pretrained VLA [1]. RoboMME [1] provides the systematic study we build on: sixteen ManiSkill [12] tasks across four memory types, and fourteen variants of $\pi_{0.5}$ crossing three representations with three integration mechanisms under a fixed 512-token budget. All fourteen variants employ a single representation; the authors note the representations are complementary and call for unified designs; this is the gap we address.

Language subgoals and their correction. Language as an intermediate plan runs from SayCan [13] and Inner Monologue [14] to RoboMME's symbolic memory, where a VLM (Gemini [15] or a fine-tuned Qwen-VL [16]) writes grounded subgoals such as "pick up the green cube at $[63, 152]$." The reliability of generated language is a known weakness: DROC [17] distills and retrieves human corrections, and YAY Robot [18] improves long-horizon performance from on-the-fly verbal feedback. Our correction signal is instead the policy's own perceptual memory, applied in latent space through a learned gate, with no human in the loop and no extra VLM call.

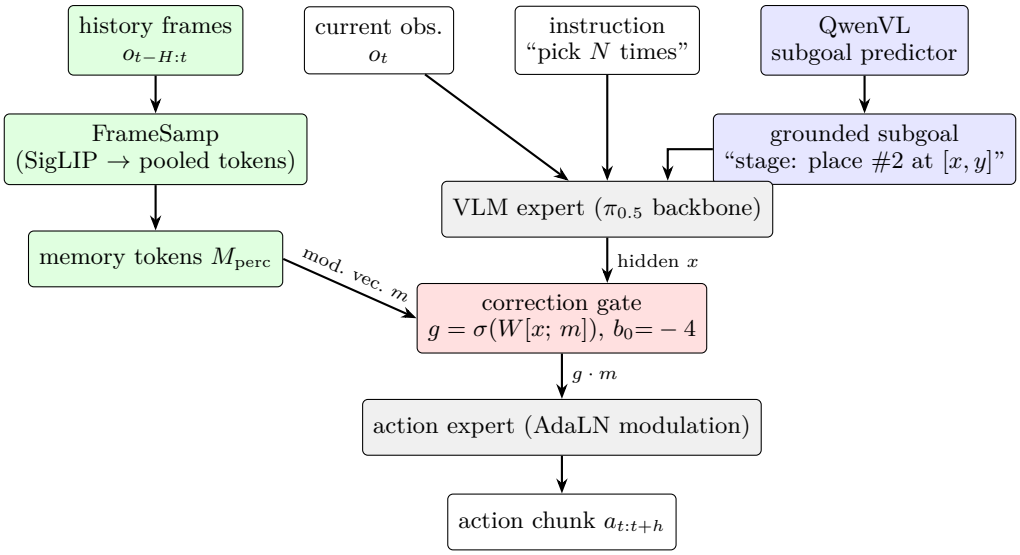

Figure 1: Dual-memory policy with perceptual correction. The symbolic pathway (blue) injects a VLM-written grounded subgoal into the prompt read by the VLM expert. The perceptual pathway (green) compresses sampled history frames into memory tokens that modulate the action expert through adaptive layer normalization [19]. The token-wise correction gate (red) computes $g \in [0, 1]$ from the hidden state and the memory modulation vector and rescales the memory's influence; its bias is initialized to -4 so memory is suppressed at the start of fine-tuning.

3 Method

3.1 Problem setting: carrying the count

PickXTimes requires the policy to track a single hidden variable: the number of completed repetitions k, compared against the instructed target N: the policy should continue while $k < N$ and press stop once $k = N$. The two memory pathways we combine carry k in opposite ways. Symbolic (explicit): an auxiliary VLM writes a grounded subgoal encoding the current stage, effectively stating k in language; the form is ideal for counting, but the value is only as reliable as the VLM that writes it. Perceptual (implicit): a frame-sampling memory retains pooled visual tokens from past observations, from which the count must be inferred; the evidence is grounded and trainable end-to-end, but after each repetition the sampled frames are nearly identical, so the inferred count is fuzzy. These two estimates of the same quantity fail differently, so a policy with both, plus a mechanism for arbitrating between them, should outperform either alone. We probe this hypothesis diagnostically, on identified failure cases (Section 4.1), rather than claiming full-distribution gains.

3.2 Dual-memory architecture

Figure 1 shows the architecture, implemented on the $\pi_{0.5}$ backbone within the MME-VLA codebase. We give each representation its strongest integration mechanism as identified by RoboMME:

Symbolic pathway (prompt context). At each control step a Qwen-VL-based subgoal predictor[1] produces a grounded subgoal, a short language string with image coordinates that encodes the current stage of the task. Because subgoals are ordinary language tokens, they are concatenated with the task instruction and consumed by the VLM expert without ar-

[1] A fine-tuned Qwen3-VL-4B, following RoboMME; we cite the Qwen2.5-VL report [16] as the closest published reference for the model family.

chitectural change. This is the natural integration for symbolic memory and costs only a handful of tokens.

Perceptual pathway (action-expert modulation). A frame-sampling module uniformly subsamples the observation history, encodes each frame with SigLIP [20], and max-pools the resulting patch grid to a small number of tokens per view under the benchmark's 512-token memory budget. The resulting memory tokens condition the action expert through cross-attention followed by adaptive layer normalization (Memory-as-Modulator), the integration RoboMME found strongest for perceptual memory: memory modulates how actions are produced without disturbing the pretrained vision–language stream.

Token-wise correction gate. The gate is the mechanism that lets one memory correct the other. For every token, given the hidden representation x and the perceptual memory modulation vector m, the gate computes

$$g \;=\; \sigma\big(W\,[\,x;\,m\,]+b\big) \in [0,1], \qquad \tilde{m} \;=\; g \cdot m, \tag{1}$$

and the rescaled vector $\tilde{m}$ replaces m in the modulation. The bias is initialized to $b_0 = -4$, so $\sigma(b_0) \approx 0.018$: at the start of fine-tuning the perceptual correction is almost silent, and the policy behaves like its symbolic parent. During training the gate learns when perceptual evidence disagrees with the symbolic stage (for example, the subgoal says "place the second cube" while the visual history shows that placement already happened) and opens on those tokens. This realizes correction in latent space with a single linear layer per block, with no extra VLM call.

3.3 Training recipe

Checkpoint merging. Training a dual policy from scratch would be wasteful: the benchmark already provides strong single-representation checkpoints. A dedicated `DualMemoryWeightLoader` merges two independently fine-tuned checkpoints (both at step 79,999), one symbolic (GroundSG) and one perceptual (FrameSamp+Modul), by loading the symbolic parameters as the base and overriding all memory-specific modules (`mem_encoder`, `mem_attn`, `mem_rms_norm`) from the perceptual checkpoint.

Adapter-only fine-tuning. With the merged initialization, we freeze the backbone and train only the lightweight components that arbitrate between pathways: the memory cross-attention adapters, their normalization layers, and the correction gate. Two configurations are used: dual-lite (2,000 steps; trains the memory adapters so the merged model learns to use both pathways together) and dual-correction (500 steps; trains the correction gate on top). Training runs on a single GPU with batch size 1, which makes the entire recipe reproducible on commodity hardware.

Stale-subgoal augmentation. The failure mode that motivates correction must appear in training data, but ground-truth subgoals from demonstrations are always correct. We therefore corrupt them in training: with a configured probability, the current subgoal is replaced by a different subgoal from earlier in the same episode (bounded lookback, duplicates filtered), simulating the drift-and-repeat behavior we observed in the real QwenVL predictor; grounded coordinates are additionally jittered with Gaussian noise to simulate imperfect grounding. Under this augmentation, the perceptual pathway is the only reliable signal for detecting that the symbolic stage is stale, which is exactly the skill the gate must learn.

3.4 Inference-time controllers: VLM-based verification

Orthogonal to the learned gate, we implemented two inference-time controllers that use explicit VLM verification, as strong baselines for what correction is worth when cost is no object. The progress-verification controller follows every QwenVL subgoal with a second VLM call that receives four recent frames, the candidate subgoal, and the expected stage sequence, and returns a verified or revised subgoal. This catches stage regressions at the

Figure 2: The PickXTimes task: a successful rollout from first pick (t=60) to pressing the stop button (t=640). The robot must pick and place the cube N times and then stop; scenes after each repetition are nearly identical, so progress must be carried by memory.

price of doubling VLM traffic. The stuck-fallback controller issues a single VLM call per step and triggers revision only when the output is unparseable, regresses to an earlier stage, or a stage exceeds its time budget; if verification also fails, an internal stage machine generates the subgoal without any VLM. Both fix symbolic errors symbolically, by asking a VLM to double-check itself; the correction gate of Section 3.2 is a learned, amortized replacement for exactly this step.

4 Experiments

4.1 Setup

Benchmark and task. All experiments run on RoboMME [1], built on the ManiSkill simulator [12] with a 7-DoF Franka Panda, front and wrist RGB cameras, and the benchmark's standard 512-token memory budget.[2] We focus on PickXTimes from the Counting suite: pick and place a colored cube N times, then press the stop button. Episodes run up to 1,300 steps (seed 7); single-memory policies are evaluated at training step 79,999, dual variants at the end of their adapter fine-tuning (Section 3.3).

Evaluation protocol. Our evaluation has two stages. Stage one runs the single-representation baselines over full episode sets (20 episodes for symbolic-QwenVL and perceptual, 10 for symbolic-oracle) to establish overall performance and to identify the failure episodes. Stage two then evaluates the dual-memory variants diagnostically on exactly those failure episodes (2, 3, and 12, with stage timeouts tightened to 96/96/64 steps for pick/place/stop), asking not "what is the average success rate?" but "does the dual design fix the specific failures that motivated it?" This protocol matches the project's goal of understanding the failure structure, at the cost of not providing a full-distribution comparison for the dual variants, a limitation we return to in Section 5.

Compared configurations. We evaluate three groups:

1. Single-representation baselines (full episode sets): symbolic GroundSG with subgoals from the fine-tuned QwenVL (realistic) and from the simulator state (oracle upper bound); perceptual FrameSamp+Modul. A no-memory $\pi_{0.5}$ baseline was not re-run in our workspace; RoboMME reports 28.8% for it on the Counting suite.
2. Dual-memory variants (failure-episode diagnostics): the merged gated model (dual-merged-gated, 500 adapter steps) and the gated model trained with stale-subgoal augmentation (dual-correction-stale, 500 steps), evaluated with oracle subgoals, with the raw QwenVL predictor, and with the QwenVL fallback controller.
3. VLM-verification controllers: the progress-verification and stuck-fallback controllers of Section 3.4 applied at inference time.

Training details. Dual fine-tuning runs for 500 steps with batch size 1 on a single GPU, updating only `mem_attn`, `mem_rms_norm`, and `mem_correction_gate`; the perceptual encoder

[2]Code and experiment logs: https://github.com/JiangHefeifei/EE265B, branch `robomem-pickxtimes-dual-memory`.

Table 1: PickXTimes results for the single-memory baselines (seed 7, checkpoint 79,999, max 1,300 steps). Failure episodes are listed explicitly because they define the targets for the dual-memory diagnostics.

Configuration	Subgoal source	Episodes	Success rate	Failed episodes
Symbolic (GroundSG)	QwenVL	20	85.0% (17/20)	2, 3, 12
Symbolic (GroundSG)	oracle	10	90.0% (9/10)	2
Perceptual (FrameSamp+Modul)	—	20	75.0% (15/20)	0, 3, 6, 11, 12

Table 2: Dual-memory diagnostic results on the identified failure episodes. These runs probe whether the dual design fixes the known QwenVL failure modes; they are not a full-distribution benchmark.

Configuration	Subgoal source	Episodes tested	Outcome
dual-merged-gated-500	oracle	2, 3, 12	2/3 (failed 2)
dual-merged-gated-500	QwenVL + progress ctrl.	2, 3, 12	1/3 (failed 3, 12)
dual-correction-stale-500	QwenVL	2, 3	1/2 (fixed 2, failed 3)
dual-correction-stale-500	QwenVL + fallback	2, 3	1/2 (fixed 2, failed 3)

and the VLA backbone stay frozen. The stale augmentation replaces the current subgoal with an earlier subgoal from the same episode with probability 0.35 (repeating the immediately previous one with probability 0.15, lookback bounded at 96 steps) and jitters grounded coordinates with Gaussian noise.

4.2 Quantitative results

Stage one: single-memory baselines. Table 1 reports full-episode results. Symbolic memory with QwenVL subgoals succeeds on 17 of 20 episodes (85%), its oracle version on 9 of 10 (90%), and perceptual memory on 15 of 20 (75%).

Two observations follow. First, on these sampled episodes the symbolic pathway outperforms the perceptual one, the opposite of RoboMME's suite-level trend (38.0% vs. 65.2% over the full Counting protocol). We attribute this to the small sample, the single task, and a well-fine-tuned QwenVL predictor that is correct on most episodes; the comparison is mainly a reminder of how wide the confidence intervals are at n=20, and of why we lean on rollout-level analysis rather than averages. Second, the QwenVL errors are concentrated: episodes 2, 3, and 12 account for every symbolic failure, and two of them (3, 12) also defeat perceptual memory. These three episodes become the diagnostic targets for the dual design.

Stage two: dual-memory diagnostics. Table 2 evaluates the dual variants on exactly the failure episodes. The headline outcomes: dual-correction-stale with QwenVL fixes episode 2 (which even oracle symbolic memory fails); episode 3 resists every variant (the symbolic sequence reaches the stop stage but low-level execution fails); and on episode 12, which no variant yet converts into an episode-level success, the perceptual-verifier controller is the only configuration whose stage decisions are correct, holding the current stage when recent frames show the pick is incomplete, where a hard timeout advances the count prematurely.

4.3 Case studies: how the symbolic pathway fails

Three episodes from the 20-episode QwenVL evaluation illustrate the failure structure that motivated, and partially validates, the dual design.

Episode 12: stale-subgoal repetition (core failure). Midway through the episode the QwenVL predictor begins re-emitting a stale first-pick or repeated-pick subgoal corresponding to a stage completed several steps earlier. The visual scene after each placement is similar enough that the VLM cannot tell the second repetition from the third; a policy that trusts the subgoal verbatim re-executes the completed stage and mis-times the stop. The error looks

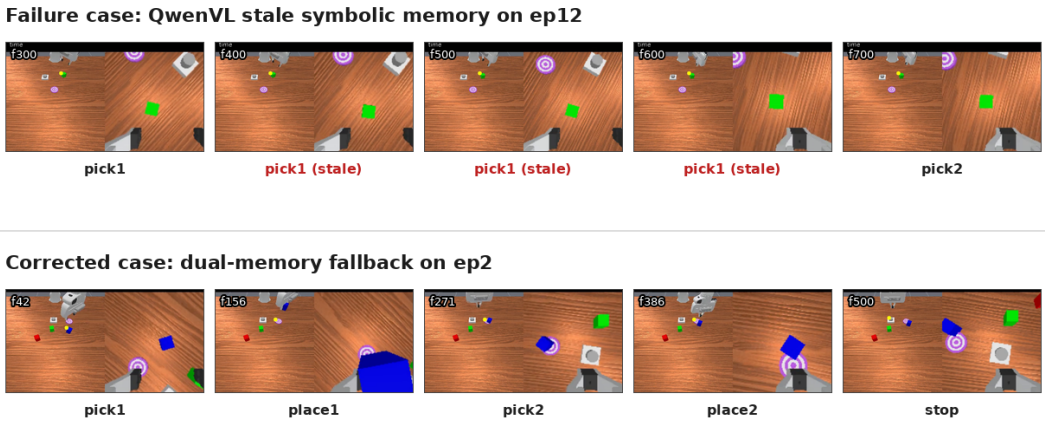

Figure 3: Case studies. Top: stale-subgoal repetition by the QwenVL predictor on episode 12: the scene advances while the predicted subgoal remains pick1 (stale) (red). Bottom: the dual-memory policy on episode 2 proceeding through the correct stage sequence to stop.

systematic rather than like an occasional hallucination: when frames are near-duplicates, the cheapest explanation for the VLM is the one it gave last time. Two remedies behave very differently. A hard timeout controller can force the symbolic sequence forward, but the advance can be premature, fixing staleness by introducing the opposite error. The perceptual verifier, in contrast, correctly held the current stage when recent frames showed the pick was not yet visually complete, advancing only on visual evidence. This contrast is the clearest single piece of evidence in the project that perceptual information is useful as a progress verifier, not as another text prompt. Our stale-subgoal augmentation is a direct simulation of this failure mode.

Episode 2: fixed by latent correction, where even the oracle fails. Episode 2 defeats not only QwenVL subgoals but also oracle symbolic memory (Table 1), so the error cannot be blamed on subgoal quality alone. The dual-correction-stale policy fixes it: with perceptual memory holding evidence of the true progress, the correction gate attenuates the conflicting modulation and the policy proceeds through the correct stage sequence. This is the intended behavior of Eq. (1), learned purely from augmented data, and it shows latent correction can repair failures that better subgoals cannot. The merged-gated variant without stale augmentation fails episode 2 even with oracle subgoals (Table 2); the fix appears only after stale-augmented training, pointing to the augmentation, not the checkpoint merge alone, as the active ingredient.

Episode 3: correction is not execution. Across every variant, the symbolic sequence can be pushed all the way to the stop stage, yet the rollout still fails; the breakdown is in low-level execution of the final manipulation, not in the count. Counting and acting remain separate competencies; memory repairs the former, not the latter. This caps achievable success regardless of how good the count is, consistent with the gap between oracle-subgoal performance and the human level reported by RoboMME.

4.4 Findings

The purpose of this project was less to maximize a success rate than to understand how a dual memory should be built. Three findings stand out.

Finding 1: VLM subgoals fail in a structured, simulatable way. The dominant error of the QwenVL subgoal predictor on counting tasks is not random noise but drift and stale repetition: re-emitting an earlier stage when consecutive observations are nearly identical

(Section 4.3). Two practical consequences follow. First, the failure can be simulated in training: our stale augmentation needs no extra data collection, only relabeling within the episode. Second, evaluation of symbolic methods should report which subgoals were wrong, not only end success: a single stale subgoal at a stage boundary is far more damaging than coordinate jitter mid-stage.

Finding 2: perceptual memory is a verification layer, not a replacement. Used alone, perceptual memory must squeeze the count out of near-duplicate frames; used against the symbolic pathway, it only needs to answer the much easier question "does the visual history contradict this subgoal?" Detecting that a placement already happened is visually simple (the gripper opened over the target; the cube returned), whereas inferring the absolute count is not. This asymmetry (verification is easier than estimation) is, we believe, the most reusable lesson of the project and the principled reason the dual design should beat both parents.

Finding 3: VLM-based verification works but does not scale. The progress-verification controller, which double-checks every subgoal with a second VLM call over recent frames, is effective at catching stage regressions, and it roughly doubles the per-step VLM cost, on top of a symbolic pathway that is already the system's latency bottleneck (RoboMME measures GroundSG-style inference at roughly 3× the backbone's compute). In practice the overhead was severe enough that we could not afford full-distribution evaluation with the verifier enabled, which is a concrete if mundane demonstration of the problem. The stuck-fallback controller recovers most of the benefit at a fraction of the calls by verifying only on anomaly triggers, and the learned correction gate goes further by amortizing verification into a single linear layer per block with zero extra calls. The progression across these three designs (verify always, verify on trigger, verify in latent space) traces the cost–reliability trade-off we expect future systems to navigate.

5 Conclusion

We presented a dual-memory VLA policy for long-horizon counting manipulation that combines symbolic subgoals and perceptual frame memory on the RoboMME benchmark, with a token-wise correction gate and a stale-subgoal augmentation that together let perceptual evidence override unreliable VLM-written subgoals. Beyond the architecture, our experiments yield a clear picture of the underlying problem: VLM subgoals fail by drifting into stale repetition; perceptual memory is most valuable as a cheap verification signal layered on an explicit symbolic state; and the cost of verification (always, on trigger, or amortized into the policy) is the axis along which such systems should be compared.

Limitations. Our evaluation covers a single task (PickXTimes); the baselines use 20 (or 10, for oracle) episodes, and the dual variants were evaluated only on the identified failure episodes, so we report no full-distribution success rate for the dual design and cannot rule out regressions on episodes the baselines already solve. On our sampled episodes the symbolic baseline outperformed the perceptual one, opposite to RoboMME's suite-level trend, which again reflects the wide confidence intervals at this scale. The correction gate was trained against simulated staleness, and although the simulation was designed from observed QwenVL behavior, the match between augmented and real failure distributions is not guaranteed. All results are in simulation on one embodiment.

Future work. The natural next step is to replace VLM-based verification entirely with a lightweight learned verifier: a small head over perceptual memory that scores subgoal consistency, trained with the same stale augmentation, and used both to gate the policy (as here) and to decide when re-querying the VLM is worth its cost. Extending the dual design beyond Counting to RoboMME's Permanence, Reference, and Imitation suites would test whether "verification is easier than estimation" holds for spatial and procedural memory as well.

Acknowledgments

This report was prepared for EE265B at UC Riverside. The implementation builds on the open-source RoboMME benchmark and MME-VLA policy-learning codebase.

References

[1] Y. Dai, H. Fu, J. Lee, Y. Liu, H. Zhang, J. Yang, C. Finn, N. Fazeli, and J. Chai. RoboMME: Benchmarking and understanding memory for robotic generalist policies. arXiv preprint arXiv:2603.04639, 2026.

[2] K. Black, A. Brohan, A. Esmail, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. arXiv preprint arXiv:2504.16054, 2025.

[3] M. J. Kim, K. Pertsch, S. Karamcheti, et al. OpenVLA: An open-source vision-language-action model. In Conference on Robot Learning (CoRL), 2024.

[4] K. Black, N. Brown, D. Driess, et al. π_0: A vision-language-action flow model for general robot control. arXiv preprint arXiv:2410.24164, 2024.

[5] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In Robotics: Science and Systems (RSS), 2023.

[6] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In Robotics: Science and Systems (RSS), 2023.

[7] H. Shi, B. Xie, Y. Liu, L. Sun, F. Liu, T. Wang, et al. MemoryVLA: Perceptual-cognitive memory in vision-language-action models for robotic manipulation. arXiv preprint arXiv:2508.19236, 2025.

[8] A. Sridhar, J. Pan, S. Sharma, and C. Finn. MemER: Scaling up memory for robot control via experience retrieval. In International Conference on Learning Representations (ICLR), 2026.

[9] T. Chen, Y. Wang, M. Li, Y. Qin, H. Shi, et al. RMBench: Memory-dependent robotic manipulation benchmark with insights into policy design. arXiv preprint arXiv:2603.01229, 2026.

[10] A. Bulatov, Y. Kuratov, and M. Burtsev. Recurrent memory transformer. In Advances in Neural Information Processing Systems (NeurIPS), 2022.

[11] Y. Sun, X. Li, K. Dalal, et al. Learning to (learn at test time): RNNs with expressive hidden states. arXiv preprint arXiv:2407.04620, 2024.

[12] S. Tao, F. Xiang, A. Shukla, et al. ManiSkill3: GPU parallelized robotics simulation and rendering for generalizable embodied AI. arXiv preprint arXiv:2410.00425, 2024.

[13] M. Ahn, A. Brohan, N. Brown, et al. Do as I can, not as I say: Grounding language in robotic affordances. In Conference on Robot Learning (CoRL), 2022.

[14] W. Huang, F. Xia, T. Xiao, et al. Inner monologue: Embodied reasoning through planning with language models. In Conference on Robot Learning (CoRL), 2022.

[15] Gemini Team, Google. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. arXiv preprint arXiv:2507.06261, 2025.

[16] S. Bai, K. Chen, X. Liu, et al. Qwen2.5-VL technical report. arXiv preprint arXiv:2502.13923, 2025.

[17] L. Zha, Y. Cui, L.-H. Lin, M. Kwon, M. G. Arenas, A. Zeng, F. Xia, and D. Sadigh. Distilling and retrieving generalizable knowledge for robot manipulation via language corrections. In IEEE International Conference on Robotics and Automation (ICRA), 2024.

[18] L. X. Shi, Z. Hu, T. Z. Zhao, A. Sharma, K. Pertsch, J. Luo, S. Levine, and C. Finn. Yell at your robot: Improving on-the-fly from language corrections. arXiv preprint arXiv:2403.12910, 2024.

[19] W. Peebles and S. Xie. Scalable diffusion models with transformers. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023.

[20] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer. Sigmoid loss for language image pre-training. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023.